INTERVIEW PREPARATION

LLM Router & Cost-Optimization Gateway

Everything to know cold before walking into an interview — what was built, why every non-obvious decision was made, and answers to the technical and behavioral questions most likely to come up.

github.com/msbhullar/llm-router
Maninderjit Singh Bhullar · Built with Claude Code

CHEAT SHEET

Quick Facts

Classifier	
Training data	21,542 labeled queries
Sources	ARC-Easy, ARC-Challenge, GSM8K, SQuAD
Model	Logistic Regression (scikit-learn)
Preprocessing	StandardScaler, class_weight="balanced"
Features	7 hand-engineered
Accuracy	87% held-out (80/20 split)
Precision/Recall	easy 0.83/0.93, hard 0.93/0.83
Inference latency	<50ms, no LLM call

Routing & Cost	
Threshold	0.3 (below = cheap, at/above = strong)
Cheap tier	gpt-5.6-luna — $0.20 / $1.20 per 1M tok
Strong tier	gpt-5.6-sol — $5.00 / $30.00 per 1M tok
Live test savings	~7% vs. always-strong baseline
Live routing split	~62% cheap / 38% strong

Backend	
Framework	FastAPI (Python)
Endpoints	`GET /`, `GET /health`, `GET /dashboard`, `POST /route`
Database	MongoDB (`routing_decisions` collection)
Logging	Async, FastAPI `BackgroundTasks`
Privacy	Only SHA-256 hash of query stored

Deployment	
Local	uvicorn, direct process
Containerized	Docker Compose (router + MongoDB)
Orchestrated	Kubernetes (Minikube)
Autoscaler	HPA, 1→3 replicas, 50% CPU target
Cloud	Documented for AWS EKS / GCP GKE, not deployed (cost)

SECTION 1

Project Overview — What We Built

The one-sentence pitch: an intelligent gateway that reduces LLM API costs by automatically routing each incoming query to the cheapest model capable of answering it correctly, instead of sending every request to the most expensive model.

The problem: most applications that call an LLM send every request to one model — usually the strongest and most expensive one, because it needs to be capable enough for the hardest queries a user might send. But most real-world queries are actually simple. Sending "what's the capital of Japan" to the same model you'd use for a multi-step reasoning problem means paying premium prices for the majority of traffic that never needed it.

The solution: a small, fast, trained machine learning classifier sits in front of two LLM tiers (a cheap/fast model and a strong/expensive model). Every incoming query is scored for difficulty in under 50ms — without ever calling an LLM to do the scoring — and routed to the cheapest tier that can handle it. Every decision is logged, and a live dashboard shows the aggregate cost savings versus a naive "always use the strongest model" baseline.

Why this matters as a portfolio project: it demonstrates real trained-ML decision-making (not a prompt-based heuristic), full-stack backend engineering (FastAPI, MongoDB, async logging), and cloud-native deployment practices (Docker, Kubernetes, autoscaling) — end to end, in one coherent system, with every phase independently verified.

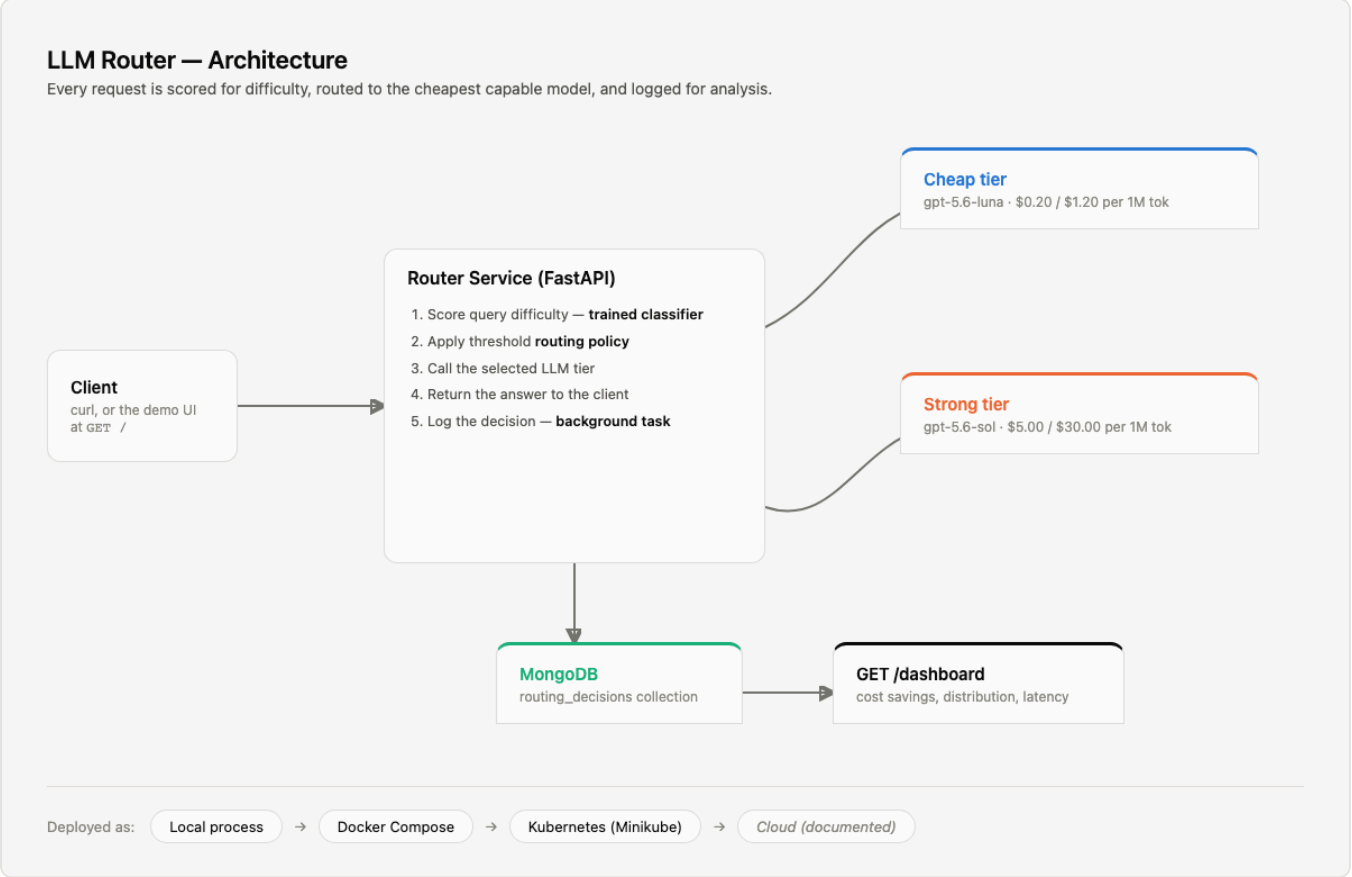
> **How it was built:** the entire system — from the ML pipeline through Kubernetes deployment — was designed, implemented, and debugged iteratively using Claude Code, an AI pair-programming tool. Each phase (classifier, backend, logging, containerization, orchestration, dashboard) was built, tested, and verified before moving to the next, following a written spec.

The Seven Build Phases

1. **Difficulty classifier** — trained offline, produces a serialized model artifact
2. **Router service** — FastAPI backend implementing the routing logic
3. **MongoDB logging** — async, fire-and-forget decision logging
4. **Containerization** — Docker + Docker Compose, one-command local stack
5. **Kubernetes orchestration** — Minikube, self-healing + autoscaling
6. **Cloud deployment** — documented (not paid-for-and-deployed) portability to AWS/GCP
7. **Dashboard & demo UI** — live cost-savings visualization and an interactive query form

SECTION 2

Architecture & Pipelines



Pipeline 1 — Offline Training (run once, produces an artifact)

`fetch_data.py` downloads and labels the 4 source datasets → `features.py` extracts the 7 features from every query → `train.py` splits 80/20, fits a `StandardScaler` + `LogisticRegression` pipeline, evaluates, and serializes the fitted pipeline to `model.joblib`. This never runs in production — it's a one-time (or occasional-retrain) offline step.

Pipeline 2 — Live Request (per `POST /route` call)

Client sends a query → `difficulty.py` loads `model.joblib` (once, at process startup) and scores the query using the exact same `extract_features()` function training used → `policy.py` compares the score against the 0.3 threshold → `llm_client.py` calls the selected OpenAI model tier → the answer returns to the client immediately → separately, a FastAPI `BackgroundTask` writes the full decision record to MongoDB *after* the response has already been sent, so logging adds zero latency to what the caller experiences.

Pipeline 3 — Dashboard (per `GET /dashboard` load)

`dashboard.py` pulls every logged document from MongoDB, computes total cost vs. a recomputed "what if this had gone to the strong model" baseline, groups by model for routing distribution, and sorts each

model's latencies in Python to compute p50/p95/p99 percentiles — all server-side, on every page load, no caching layer.

Pipeline 4 — Deployment (same code, four environments)

The exact same `router_service` code runs unmodified as: a local `uvicorn` process → a Docker Compose stack → a Kubernetes deployment on Minikube → (documented) a real cloud cluster. The *only* thing that changes between environments is the `MONGODB_URI` environment variable — `localhost` for local dev, `mongo` (resolved via each platform's own service discovery) for both Docker Compose and Kubernetes.

Critical coupling point: train/serve skew

The single most important architectural decision in this system is that `classifier/features.py`'s `extract_features()` function is imported *directly* by both the training script and the live router — not reimplemented twice. If feature computation ever diverged between training and serving, the model would silently receive different inputs than it was trained on (train/serve skew), degrading accuracy without any visible error.

SECTION 3

Tech Stack & Why Each Choice

Technology	Why it was chosen
Python	The default choice for the ML side (scikit-learn/spaCy ecosystem) and pairs naturally with FastAPI for the backend — one language across the whole system.
FastAPI	Async-native, automatic request/response validation via Pydantic models, minimal boilerplate, and it's the modern standard for Python APIs — directly relevant since the project needed a fast, typed JSON API.
scikit-learn	Mature, well-documented, and gives an interpretable model (see Section 4) with a simple `Pipeline` abstraction for chaining scaling + classification.
spaCy	Production-grade NLP library, used specifically for its named-entity recognizer to compute the `entity_count` feature.
wordfreq	A lightweight library giving real-world word-frequency data, used to compute `rare_word_ratio` — a domain-agnostic proxy for jargon/technical density.
MongoDB	Schema-flexible document store — a natural fit for logging structured-but-evolving decision records, and it demonstrates NoSQL alongside the project's other relational-adjacent skills.
Docker / Docker Compose	Reproducible packaging and one-command local multi-service orchestration (router + MongoDB together).
Kubernetes	The industry-standard container orchestrator — chosen specifically to demonstrate self-healing deployments and CPU-based autoscaling, not just "the app runs in a container."
OpenAI API	The two LLM tiers being routed between. (See the callout below — this was originally planned as Anthropic/Claude.)
Claude Code	AI pair-programming tool used to iteratively design, build, test, and debug the entire system across all seven phases.

> **Why OpenAI and not Anthropic?** The original plan was Claude (Haiku as the cheap tier, Sonnet as the strong tier) — the classifier and routing logic are fully provider-agnostic, so switching providers was just a small, isolated change to `llm_client.py` and `config.py`. The actual reason for switching was practical: the Anthropic Console's billing "buy credits" button was stuck in a disabled state during setup, and a funded OpenAI account was already available. Both providers require a separate paid API account, distinct from any consumer chat subscription — so this wasn't about avoiding that step, just a case where one provider's billing was already sorted. **Good interview answer if asked "why OpenAI":** be honest about this — it demonstrates that the system's design (provider-agnostic core, provider-specific adapter) made a real-world pivot cheap and low-risk, which is itself a good architecture story.

SECTION 4

Deep Dive: The Three "Why" Questions

Why Logistic Regression (and not a neural net, gradient boosting, or an LLM-as-judge)?

- **Interpretability was not optional — it was actively used.** When an early version of the classifier looked suspiciously good (96% accuracy) but failed on realistic queries, inspecting the logistic regression's coefficients directly revealed the problem: one feature (`entity_count`) had roughly 4x the weight of any other, exposing that the model had learned a dataset-format shortcut rather than real difficulty. A black-box model (small neural net, gradient boosting ensemble) would have hidden this — there'd be no simple coefficient table to inspect, only much more expensive techniques like SHAP values.
- **Latency requirement.** The classifier has to run in well under 50ms with no LLM call, since it sits on the critical path of every request before the actual (expensive, slow) LLM call happens. Logistic regression inference is a single matrix multiply — effectively instant. A more complex model wouldn't meaningfully improve accuracy here (the bottleneck was never model capacity — see below) while adding latency risk for no benefit.
- **Data/feature scale doesn't justify more capacity.** With 7 engineered features and ~21,500 rows, this is a small, well-behaved tabular classification problem. A deep learning model would be solving a problem it's overkill for, and would risk overfitting on a feature set this small.
- **This was tested, not assumed.** Gradient boosting was actually tried as an experiment on the same features, and it performed essentially the same as logistic regression (57% accuracy, versus logistic regression's 57%, on the early ARC-only attempt). That result was itself informative — it confirmed the accuracy ceiling was a data/feature problem, not a model-capacity problem, which is exactly what led to fixing the dataset instead of trying yet another model.
- **Why not an LLM-as-judge for difficulty scoring?** That would defeat the entire point of the project — the whole system exists to avoid an LLM call on every request. Scoring difficulty with an LLM would add the exact latency and cost the router is designed to eliminate.

Why this dataset (and why it changed twice)?

This is the single best story in the project to tell in an interview — it shows genuine debugging and ML judgment, not just "I trained a model and it worked."

- **Attempt 1 — ARC-Easy vs. ARC-Challenge only: 57% accuracy.** Barely above a naive baseline. Root cause: ARC's difficulty split is about scientific-knowledge depth, which isn't visible in surface-level text features like length or keywords — no amount of feature engineering on *text structure* can recover a difficulty signal that's actually about *domain knowledge*.
- **Attempt 2 — ARC-Easy vs. GSM8K: 96% accuracy, and wrong.** This looked like a huge win on paper. But testing the model on hand-written, realistic queries it had never seen (not sampled from either training dataset) revealed it called *almost everything* "easy" — including genuinely complex comparison questions — while a bare "hi" scored nearly as "hard" as a real question. Inspecting the model's coefficients showed why: `entity_count` dominated every other feature, because GSM8K's word-problem narratives ("Tom has 5 apples...") are naturally entity-rich (names, quantities) while ARC's terse science-exam stems are not. The model had learned "does this look like a GSM8K-style story problem," not "is this hard."

- **The fix — blend multiple genres into *both* classes:** the final dataset mixes ARC-Easy + SQuAD (easy) against GSM8K + ARC-Challenge (hard), specifically so that both classes contain entity-rich and entity-sparse examples. This breaks the genre-as-shortcut signal: entity count can no longer perfectly predict the label, so the model is forced to rely on features that actually correlate with difficulty. Accuracy dropped to a more honest 87% — a decrease that reflects removing a shortcut, not a regression.
- **The general lesson (great to state explicitly in an interview):** a high test-set score doesn't guarantee real generalization if the two classes differ systematically in *writing style*, not just difficulty. The only way to catch this is testing against realistic, out-of-distribution examples — not just trusting the held-out split, which is drawn from the same biased distribution as the training data.

Why these specific 7 features?

Design constraint behind every one: must be computable in well under 50ms, deterministic, and require zero LLM calls.

Feature	Purpose
`word_count`	Longer queries often require more context or reasoning to answer fully.
`reasoning_keyword_count`	Explicit signal phrases ("compare," "explain why," "step by step") indicate the user wants multi-step reasoning, not a lookup.
`question_mark_count`	Proxy for multiple distinct sub-questions bundled into one query.
`conjunction_count`	Proxy for multiple clauses/sub-questions joined together ("and," "or," "but").
`entity_count` (spaCy NER)	Originally intended as a complexity signal. **Known limitation (good to volunteer honestly):** in practice this measures named-entity density more than true difficulty, and was the exact feature responsible for the dataset-shortcut bug above — it's kept because it still adds some signal once the shortcut is broken, but it's the weakest-justified feature in the set.
`avg_word_length`	Loose proxy for vocabulary complexity.
`rare_word_ratio` (via `wordfreq`)	The most deliberately-designed feature: measures what fraction of a query's words fall below a common-usage frequency threshold (Zipf < 3.5) — a domain-agnostic way to detect technical/jargon-heavy language, whether the jargon is science, law, or finance. **Known limitation:** can't distinguish word senses, so dual-meaning words ("cellular" as in biology vs. phones) sometimes score as common even in a technical context.

SECTION 5

15 Technical Interview Questions

1 Walk me through this project end to end.

It's a cost-optimization gateway that sits between a client and two LLM tiers. A query comes into a FastAPI backend, gets scored for difficulty by a logistic regression classifier I trained (in under 50ms, no LLM call involved), and based on a threshold, gets routed to either a cheap or a strong OpenAI model. The answer returns immediately to the client, and in the background — without adding latency — the full decision (difficulty score, model used, cost, latency) gets logged to MongoDB. A dashboard reads that log and shows aggregate cost savings versus always using the strong model. The whole thing is containerized with Docker and deployed to Kubernetes with a working autoscaler, and I have it documented (though not paid-for-deployed) for a real cloud cluster.

2 Why did you choose logistic regression over a more complex model?

Three reasons. First, interpretability wasn't optional — I actually used coefficient inspection to diagnose a real bug where the model had learned a dataset shortcut instead of real difficulty; a black-box model would have hidden that. Second, latency — the classifier runs on the critical path before every LLM call, so it has to be near-instant, and logistic regression inference is a single matrix multiply. Third, I tested this empirically: I tried gradient boosting on the same features and got essentially the same accuracy, which told me the bottleneck was the data and features, not model capacity, so a more complex model wouldn't have helped.

3 Tell me about the dataset-shortcut bug you found. How did you catch it, and how did you fix it?

I had a classifier scoring 96% test accuracy, which looked great. But when I ran it on hand-written queries that weren't from either training dataset, it called almost everything "easy" — even genuinely hard comparison questions — while a bare "hi" scored nearly as "hard" as a real question. I inspected the logistic regression's coefficients and found one feature, named-entity count, had about 4x the weight of any other. The root cause: my "hard" examples came from GSM8K math word problems, which are naturally entity-rich (names, quantities), while my "easy" examples were terse science questions with almost no entities. The model had learned "does this look like a GSM8K story problem," not "is this hard." I fixed it by blending multiple genres into both classes — adding SQuAD to the easy side and ARC-Challenge to the hard side — so entity density could no longer perfectly predict the label. Accuracy dropped to a more honest 87%, which I consider a better result because the shortcut was gone.

4 How do you prevent train/serve skew in this system?

The feature extraction function lives in exactly one place, `classifier/features.py`, and both the training script and the live router import it directly — neither reimplements feature computation independently. That's a deliberate architectural choice: if training-time and serving-time feature logic ever diverged, the model would silently receive different inputs in production than it was trained on, degrading accuracy without any visible error. Sharing the exact same code eliminates that risk by construction rather than by discipline.

5 Why a simple threshold for routing instead of something more sophisticated?

The classifier outputs a continuous 0–1 probability, not a binary label, and the routing decision is a separate, isolated function from the scoring itself. That separation means the "simple threshold" is actually a cheap starting point I can extend without retraining anything — going from two tiers to three just means adding a second threshold in the same function. I kept it simple because the spec's own acceptance criteria didn't require more, and building a fancier rules engine before I had real usage data to justify it would have been premature complexity.

6 How does the cost-savings calculation actually work?

On every request, regardless of which tier actually handled it, I compute two numbers from the real token counts OpenAI returns: the actual cost at the tier that was used, and a "baseline cost" — what this exact query would have cost if it had gone to the strong tier instead, using the same token counts. I log both to MongoDB on every request. The dashboard sums both fields across all logged requests and reports the difference as total savings. It's an approximation, since a different model might have produced a different-length response, but using the real token counts from the actual response is the closest reasonable estimate.

7 Why MongoDB instead of a relational database?

Each routing decision is a self-contained, flat document — timestamp, query hash, difficulty score, model used, costs, latency, token counts — with no relationships to other records that need to be queried or joined. MongoDB's document model fits that shape naturally without needing a schema migration if I add a field later. It also let me demonstrate NoSQL experience alongside the project's other database work. For this specific access pattern — write-once records, read-and-aggregate for the dashboard — a relational database wouldn't have added anything a document store doesn't already provide simply.

8 How do you make sure logging doesn't slow down the actual API response?

I use FastAPI's `BackgroundTasks`, which runs the log write *after* the response has already been sent to the client — not before, and not blocking the request. That's the specific mechanism the spec's "fire-and-forget, asynchronous" logging requirement was asking for. I considered a fully async database driver (motor) instead, but for this project's scale, adding a whole async database layer would have been unnecessary complexity — BackgroundTasks does the actual job without it.

9 Walk me through your Kubernetes setup. What resources did you use and why?

I have a Deployment and Service for MongoDB (with a PersistentVolumeClaim so data survives pod restarts), and a Deployment, Service, and HorizontalPodAutoscaler for the router. Non-secret configuration — model names, the routing threshold — lives in a ConfigMap, version-controlled as YAML; the OpenAI API key lives in a Secret, created by piping just that one line from my .env file directly into kubectl, so the key value is never written into a YAML file or displayed anywhere. The HPA scales the router from 1 to 3 replicas based on CPU utilization crossing 50% of its requested 200 millicores — that requested-CPU value is required for the HPA to have a baseline to compute a percentage against.

10 Why didn't you deploy this to a real AWS or GCP cluster?

Both incur ongoing costs the moment a cluster exists — AWS EKS has a mandatory ~$0.10/hour control-plane fee with no free-tier exemption, and GCP GKE's free zonal cluster still needs paid VM nodes underneath it. For a portfolio project with no real traffic, paying to keep infrastructure running 24/7 isn't a good tradeoff. What I did instead: every Kubernetes manifest is written to be portable — the only changes needed for a real cloud cluster are pushing the image to a registry instead of loading it locally, and changing the Service type from ClusterIP to LoadBalancer for a public IP. I have that migration path fully documented. I'd rather be honest about a deliberate cost decision than claim cloud experience I don't have — and if asked to actually demo it live, I could spin up GKE on free trial credit in about 30 minutes.

11 How would you extend this to support a third model tier?

Because the classifier already outputs a continuous 0–1 score rather than a binary label, adding a third tier doesn't require retraining anything — it means adding a second threshold to the routing policy function, e.g. below 0.3 goes cheap, 0.3–0.7 goes mid-tier, above 0.7 goes to the strongest model. The routing decision was deliberately kept as an isolated, single-purpose function specifically so this kind of policy change wouldn't touch the classifier, the API layer, or the logging — only that one function.

12 What are the limitations of your classifier, and how would you improve it for production?

Two honest limitations. First, it's trained on science, math, and trivia-style benchmarks — a production system fielding coding questions, creative writing, or business reasoning would need broader domain coverage in the training data. Second, the named-entity-count feature is weaker than its name suggests — it measures proper nouns, not general technical vocabulary, and was the exact feature responsible for the dataset-shortcut bug I found. For production, I'd want to add the embedding-similarity feature the original spec mentioned as optional — comparing a new query's embedding against a labeled reference set — which I deliberately deferred for v1 because it requires standing up a whole embedding-and-vector-lookup pipeline for one feature.

13 What happens if the OpenAI API call fails, or MongoDB is down?

Honestly, right now the system doesn't handle either gracefully — an OpenAI failure propagates as a 500 error to the client, and the code doesn't currently have retry logic or a fallback model. That's a real gap I'd flag rather than paper over. For a production version, I'd add retry-with-backoff on the OpenAI call, and since the MongoDB write happens in a background task after the response is already sent, a MongoDB outage wouldn't break the user-facing request — it would just mean that decision doesn't get logged, which I'd want to catch and alert on rather than silently lose.

14 Why did you store only a hash of the query instead of the raw text?

The spec's own logging schema explicitly called for avoiding unnecessary PII storage, so I hash the query with SHA-256 before writing it to MongoDB. That's still enough to detect duplicate or repeated queries for analysis purposes, without retaining what a user actually typed. It's a small design choice, but it reflects a habit worth having by default — not storing more than the system actually needs.

15 How would this system need to change to handle real production traffic at scale?

A few things. The dashboard currently pulls every logged document from MongoDB and aggregates in Python on every page load — fine at hundreds of rows, but I'd move to a proper MongoDB aggregation pipeline (or a pre-computed rollup) at real scale. The BackgroundTasks logging approach is single-process; at high throughput I'd likely move to a proper message queue so logging failures can't pile up inside the API process. And I'd want real retry/circuit-breaker logic around the OpenAI calls, plus the embedding-similarity feature and broader training-data domain coverage mentioned above.

SECTION 6

10 Behavioral Questions

1 Tell me about a time you found a bug that wasn't obvious.

Situation: I'd just trained a difficulty classifier that scored 96% accuracy on its held-out test set — a great-looking number. **Task:** before trusting it, I wanted to sanity-check it against realistic queries. **Action:** I ran it on a small set of hand-written questions that weren't from either training dataset, and the results were wrong in an obvious way once I looked — it called almost everything "easy," including genuinely hard comparison questions, while a plain "hi" scored nearly as high as a real question. I inspected the model's coefficients directly and found one feature dominating all the others by about 4x, which pointed me straight at the root cause: the model had learned to detect which dataset a question came from, not how hard it actually was. **Result:** I fixed the training data to remove that shortcut and landed on a more honest 87% accuracy. The lesson I took from it: a good test-set number doesn't prove generalization if you haven't checked it against data the model has never implicitly memorized the shape of.

2 Tell me about a time your initial approach didn't work and you had to pivot.

Situation: my first attempt at the classifier used only two difficulty tiers from the same benchmark (ARC-Easy vs. ARC-Challenge). **Task:** get a classifier that could actually distinguish difficulty. **Action:** that first attempt only reached 57% accuracy — barely better than guessing. Rather than immediately trying a more complex model, I first tried swapping the algorithm (gradient boosting instead of logistic regression) to rule out "the model isn't powerful enough." It performed almost identically, which told me the real problem was the data, not the model. **Result:** I pivoted to a completely different dataset combination, which is what eventually led to the shortcut bug in the previous story, and then to the real fix. The pivot itself taught me to test which layer of the system is actually the bottleneck before assuming I know the answer.

3 Describe a time you had to make a tradeoff between the "ideal" solution and a pragmatic one.

Situation: the project spec called for demonstrating cloud deployment on AWS or GCP. **Task:** decide whether to actually provision and pay for live cloud infrastructure. **Action:** I weighed the cost — both providers charge for a running cluster the moment it exists — against the fact that this project has no real production traffic to justify that ongoing spend. I chose to fully write and document the Kubernetes manifests as portable to a real cloud cluster, verified locally on Minikube, and documented exactly what would change to deploy for real, rather than actually paying to keep something running that nobody was using. **Result:** I get to be upfront about that decision rather than either lying about having deployed it, or spending money for no real benefit — and I can spin it up live on free trial credit in about 30 minutes if a specific situation calls for it.

4 Tell me about a time you debugged something that looked like a code problem but wasn't.

Situation: after adding a new dashboard endpoint and rebuilding my Docker image, I tested it and got a 404 — the endpoint didn't exist as far as the server was concerned, even though I could see it clearly in the source file. **Task:** figure out why confirmed-correct code was producing wrong behavior. **Action:** I ruled out Docker layer caching by doing a full no-cache rebuild, which didn't fix it — a strong signal the code itself wasn't the problem. I checked what process was actually listening on that port and found a `kubectl port-forward` process from an earlier Kubernetes testing session still running in the background, silently intercepting the port and routing my requests to an old Kubernetes pod instead of my new Docker container. **Result:** killing that leftover process fixed it immediately. The general lesson: when code you've verified is correct still produces wrong behavior, check what's actually receiving the request before assuming the code is wrong again.

5 Describe a situation where you had to learn something new quickly to get the job done.

Situation: the project required deploying to Kubernetes, which I hadn't worked with hands-on before this project. **Task:** get a working Deployment, Service, ConfigMap, Secret, and autoscaler running, understanding what each piece actually did rather than just copying manifests. **Action:** I built it up incrementally — MongoDB first, verified it came up correctly, then the router, then the autoscaler — testing at each step rather than writing everything at once and debugging a pile of failures together. I hit real gotchas along the way (Minikube not sharing the host's Docker images, the autoscaler needing an explicit CPU resource request to have a baseline to measure against) and had to understand why each one happened, not just paste a fix. **Result:** I came out with a genuinely working, self-healing, autoscaling deployment, and I can explain every piece of it, not just that it runs.

6 Tell me about a decision you'd reconsider or do differently if you started over.

Situation: the named-entity-count feature in my classifier turned out to be the exact feature responsible for the dataset-shortcut bug — it wasn't measuring what I intended. **Task:** decide whether to keep it, replace it, or drop it. **Action:** I kept it, since once the underlying dataset shortcut was fixed, it still contributed some real signal alongside the other six features — but I was explicit in my own documentation that it's the weakest-justified feature in the set, since it really measures proper-noun density more than general technical complexity. **Result:** if I were starting over, I'd either replace it with a better technical-vocabulary signal from the start, or design the "hard" and "easy" verification checks earlier in the process instead of after training looked successful — catching a shortcut before trusting a good-looking number, rather than after.

7 Tell me about a time a result looked good but you didn't just accept it.

Situation: this is the same classifier story from a different angle, worth having ready as a distinct answer. My model hit 96% test accuracy, which by any normal metric is an excellent result. **Task:** decide whether to ship it or dig further. **Action:** instead of accepting the number, I deliberately tested it against examples it had never seen the shape of — not because I suspected a specific problem, but because a held-out test set drawn from the same biased data can't catch a shortcut baked into that same data. That test immediately exposed the model was badly wrong in practice despite the great score. **Result:** I've since made this a standing habit in the project — there's a permanent sanity-check script that runs the classifier against hand-written, realistic queries after every retrain, specifically because a single accuracy number is not sufficient evidence on its own.

8 Describe a time you had to balance speed/simplicity against doing something more "correct."

Situation: MongoDB in my Kubernetes setup runs as a plain Deployment with a PersistentVolumeClaim, not a StatefulSet — which is the more "correct" Kubernetes primitive for databases in production, since it gives stable network identity and ordered, graceful scaling. **Task:** decide whether that mattered here. **Action:** I explicitly reasoned through it: a StatefulSet's guarantees matter for multi-replica databases where pod identity and startup order need to be predictable. At this project's scale — a single-replica MongoDB instance — those guarantees add complexity without adding real benefit. **Result:** I used the simpler primitive and documented why, rather than either over-engineering for a scale I don't have, or blindly copying a "best practice" without understanding when it actually applies.

9 **Tell me about a time you worked with ambiguous or underspecified requirements.**

Situation: the original project spec described the system at a fairly high level — it named the general approach (a trained classifier, a threshold policy, Kubernetes deployment) but left many concrete choices open: which datasets, which exact features, what threshold value, which cost rates. **Task:** turn that into real, defensible engineering decisions rather than arbitrary ones. **Action:** for each open decision, I reasoned through the actual tradeoffs before picking — for example, choosing the routing threshold and dataset mix based on what I could verify empirically, not just picking round numbers. When a choice was genuinely a judgment call with real tradeoffs (like the cloud deployment decision), I made the call explicitly and documented the reasoning rather than leaving it implicit. **Result:** every non-obvious decision in the system has a "why" behind it I can explain, which matters more in an interview than just having followed a spec literally.

10 **How do you approach validating your own work, especially when there's no one else reviewing it?**

Situation: working solo on this project meant I was the only check on whether something actually worked versus just looked like it worked. **Task:** build a habit of verifying claims rather than trusting them. **Action:** a few concrete habits came out of this project specifically: never trusting a held-out accuracy number alone (see the sanity-check story above), always testing a live request end-to-end after a deploy rather than assuming a successful build means a working app, and re-running the actual training script to re-verify numbers before writing them into documentation rather than trusting my memory of an earlier run. That last habit caught a real error — I'd been saying my classifier used "6 features" throughout my documentation and even my resume, when the code actually had 7; re-checking the source directly caught it. **Result:** the project's documentation is something I trust because I've verified it against the running system, not just written it once and assumed it stayed accurate.